

Serverless computing, FaaS model

Serverless computing and the Function-as-a-Service (FaaS) model are the next stage in cloud computing. **They represent a shift away from traditional server-based computing models towards a more dynamic, scalable, and event-driven architecture.**

By hearing the term Serverless Computing most of us may misunderstand that we are entering into a new era of computing, where we don't require a server to run the applications. Even though the term is a misnomer, serverless computing portrays a new stage in cloud maturation. This feature is dedicated to those developers who are keen on knowing about the next level of cloud computing. Let's dig deep into the following sections.

- Serverless Computing
- FaaS Model
- Advantages and Limitations of Serverless Computing
- AWS Serverless Platform

Here are some key features and benefits of serverless computing and the FaaS model:

1. **No Server Management:** With serverless computing, developers **do not have to worry about server management, provisioning, or scaling**. The cloud provider takes care of these tasks, allowing developers to focus on writing code and building applications.
2. **Pay-Per-Request Pricing:** Serverless computing follows a pay-per-request pricing model, which means that developers only pay for the resources they use, rather than paying for an entire server or virtual machine.
3. **Event-Driven Architecture:** Serverless computing is built on an event-driven architecture, which means that **applications are triggered by events, such as user actions or system events**. This allows for a more dynamic and responsive computing environment.
4. **Scalability:** Serverless computing is highly scalable, as resources are automatically provisioned and deprovisioned based on demand. This allows applications to scale up or down quickly and efficiently.
5. **Faster Development:** With serverless computing, developers can build applications more quickly and easily, as they do not have to worry about server infrastructure or deployment.
6. **Reduced Costs:** Serverless computing can help to reduce costs, as developers only pay for the resources they use, and do not have to worry about managing server infrastructure.

However, there are also some challenges and limitations with serverless computing, such as limited control over the underlying infrastructure, lack of flexibility, and potential vendor lock-in. It is important for developers to carefully consider these factors when deciding whether to adopt a serverless computing approach.

Overall, serverless computing and the FaaS model represent an exciting new direction for cloud computing, offering a more dynamic, scalable, and event-driven architecture that can help to accelerate application development and reduce costs.

Serverless Computing

Serverless Computing is a cloud computing execution model that provides flexibility to develop, deploy, and run applications without worrying about provisioning servers. **It helps developers to focus on their core product instead of spending time to provision, scale, and manage servers.**

This cloud computing execution model doesn't eliminate the need for a server. But it provides a complete infrastructure that requires to run your application by adding a layer of abstraction above the cloud infrastructure. Some of the cloud service providers that provide a serverless framework are AWS, Microsoft Azure, and the Google Cloud Platform (GCP).

Features of serverless computing

- Serverless computing allows you to **run code without managing servers**, and the functions are executed based on an event rather than running all the time.
- You can **run any type of application irrespective of the languages** or the frameworks to be used.
- It **facilitates the automatic scaling of a function based on the request**. You don't need to do any further configuration.
- The functions can be triggered by different event types and as well as using API gateways.
- FaaS Model

The best way to understand serverless computing is by explaining the FaaS model. Most of us are familiar with the terms SaaS, IaaS, and PaaS. Let's have a look at these terms before we get into FaaS.

Software as a Service (SaaS) is a cloud computing service where software is hosted centrally, and the service is delivered through the internet based on a subscription. Some of the examples of SaaS are Google Apps, Dropbox, WebEx, and so on.

Infrastructure as a Service (IaaS) provides infrastructure resources and also manages and scales up the resource. Here you only pay for what you use. A few of the examples of IaaS are AWS EC2, Azure virtual machines, and so on.

Platform as a Service (PaaS) provides both software and infrastructure that require for the application development. AWS Beanstalk, Azure app services are some of the examples.

So, let's get into the FaaS model.

It is a **Function as a Service** model that allows the developers to develop, run, and manage applications without worrying about the application infrastructure.

It is based on the concept of the single-purpose block of code called **functions**. Here, the model uses an **event-driven architecture** to process these functions, in which each function gets executed by triggering an event - such as API requests, events in the database, scheduled events, etc. And, serverless applications are built by combining multiple functions (FaaS).

Another point to take into account related to the FaaS model is that it is **stateless**. Since functions in the FaaS model runs in transient containers, you will not be able to access state from a previous run. However, you can use external sources such as Amazon S3 to store and retrieve the data instances. This data can be shared within the functions.

If you look carelessly, you may feel like PaaS and FaaS are almost similar. But a detailed analysis will help you to differentiate between PaaS and FaaS. Let's take a moment to understand the difference between them.

- Even though PaaS offers easiness in deploying and managing web applications, it doesn't provide the flexibility to take your entire application up and down for each request. On the other hand, FaaS provides the flexibility to do the same.
- In PaaS infrastructure, you have to depend on other PaaS based solutions such as AWS Elastic Beanstalk to handle application scaling. But these solutions are not flexible enough to deal with individual requests, whereas FaaS infrastructure offers efficient ways to handle application scaling.

Advantages and Limitations of Serverless Computing

So far, we have discussed serverless computing and FaaS model. Now, let's look at the advantages and limitations of Serverless computing.

Advantages of Serverless Computing

Serverless computing offers numerous benefits. Let's look at a few of them.

1. **Reduced operational cost:** In FaaS, pre-defined runtimes (infrastructure is only used for a certain period) and sharing the same runtimes contributes to a significant reduction in operational cost.
2. **Rapid development:** Since the infrastructure is managed by the cloud provider, the developers can concentrate on the core features.
3. **Scaling costs:** It facilitates automatic horizontal scaling and also takes care of scaling up and scaling down the infrastructure. This offering results in a significant reduction in scaling cost, when compared with the PaaS offering.
4. **Easier operational management:** The FaaS provides the simplest solution in deploying and managing applications. Above all, you can transform your business idea into a reality within a short period.

Limitations of Serverless Computing

Even though serverless computing has numerous benefits, there are certain limitations as well. Let's have a look at the below points.

1. **Infrastructure control:** Since serverless architectures are monitored by the cloud providers, you don't have any control over the infrastructure.
2. **Long-running application:** It is not well adapted for long-running batch operations because of its pre-defined runtime features, where most cloud providers have a timeout period.
3. **Vendor lock-in:** One of the major limitations is vendor lock, where you cannot switch to another cloud provider from the existing one.
4. **Cold start:** Since FaaS is an event-driven architecture, it raises the concern of a cold start. It may take a bit longer to respond to an event after a period of inaction.
5. **Shared Infrastructure:** Since the serverless application uses shared infrastructure, many applications can run alongside irrespective of business ownership. So there are chances that your code might be affected by high load generating functions of the neighboring applications. But this problem exists not only in serverless offerings but also in many other shared service offerings as well.

Disadvantages:

1. **Lack of Control:** With serverless computing, developers have limited control over the underlying infrastructure, which can be a disadvantage in some scenarios. They have to rely on the cloud provider for managing and maintaining the infrastructure.
2. **Limited Flexibility:** Serverless computing may not be suitable for all types of applications or workloads, as it requires a specific architecture and design approach.
3. **Cold Start Latency:** Serverless functions may experience a delay in starting up if they have not been accessed for a while, which can impact application performance.
4. **Potential Vendor Lock-In:** Serverless computing can create a dependency on a specific cloud provider, which can limit flexibility and make it difficult to switch to a different provider.

Overall, the advantages of serverless computing and the FaaS model outweigh the disadvantages for many use cases, but it is important for developers to carefully consider the trade-offs when deciding whether to adopt this approach.

AWS Serverless Platform

AWS provides a fully managed serverless platform to deploy and run your application. Let's look into the services provided by AWS serverless platform.

Compute: AWS provides services such as [AWS Lambda](#) to run the code without provisioning or managing servers, [Lambda@Edge](#) to respond to events generated by Amazon CloudFront events, and [AWS Fargate](#) facilitates a serverless compute engine for containers.

Storage: AWS provides [Amazon Simple Storage Service](#) (Amazon S3) that ensures secure, durable, highly-scalable **object storage**, and [Amazon Elastic File System](#) (Amazon EFS) provides simple, scalable, elastic **file storage**.

Data Stores: [Amazon DynamoDB](#) provides NoSQL database service, and [Amazon Aurora Serverless](#) (for [Amazon Aurora](#)) permits automatic database start-up, shut down, and scale capacity up or down. AWS also offers database proxy services using [Amazon RDS Proxy](#).

API Proxy: [Amazon API Gateway](#) service helps developers to create, publish, maintain, and secure APIs at any scale. Here, it allows you to process hundreds of thousands of concurrent API calls by efficiently handling traffic.

Application Integration: It facilitates services such as [Amazon SNS](#) (pub or sub messaging service), [Amazon SQS](#) (message queuing service), [AWS](#)

[AppSync](#) (to securely access, manipulate, and combine data), and [Amazon EventBridge](#) (to transfer application data from different sources to your AWS environment).

Orchestration: [AWS Step Functions](#) orchestrates the components of distributed applications and microservices.

Analytics: AWS provides services such as [Amazon Kinesis](#) (for streaming data on AWS) and [Amazon Athena](#) (an interactive query service to analyze data) to deal with data analytics.

Apart from the above services, it also provides a wide array of [tools](#) for the serverless application development process. These tools include continuous integration and testing tools, deployment, monitoring, and diagnostics tools, frameworks, and IDE plugins.

Summary

Serverless computing offers significant benefits such as rapid development, serverless deployment, lesser operational cost, and easier operational management. However, it doesn't mean No Ops. Indeed, you don't need a system administrator to manage the infrastructure, but you should monitor certain operations to ensure that your application is running. But no worries, FaaS itself provides monitoring data services that help you to monitor your applications. And one final point to note, it may not be the right approach for every challenge, so be brilliant enough to choose it wisely. Get ready to harness the serverless technology, and concentrate on the coding process rather than provisioning servers.

DevOps principles

In recent years due to the advancement in technologies, **DevOps**, short for **Development and Operations** has emerged as a transformative approach that mainly streamlines **software delivery**, helps in enhancing collaboration, and accelerates the time-to-market. Thus **DevOps is not just a tool within an organization but it is a cultural and technical movement that has a motive to break down the silos between the development and operations teams.**

What is DevOps?

DevOps is a set of principles that are the fundamental groundwork of the culture of **DevOps**. The main objective of the DevOps methodology is to speed up the time to market apply the incremental improvements in response to the changing environment and develop a more streamlined

development process. Therefore in simple terms, DevOps is one of the fields in engineering that is responsible for the designing implementation, and management of [CI/CD](#) frameworks.

DevOps refers to the integration of the operation and the development of teams which means that collaboration is the major part of DevOps.

Therefore by working together, the development team can better configure the software for the operating phase and the operations to test the software to make sure that it fulfills the needs. **Collaboration also depends on the information sharing practices which is an issue** that is discovered while the applications are being deployed and should be adequately logged so that the other development team can account for this for future purposes.

2. Customer-Centric Decision Making

Customer-centric decision-making is an important DevOps principle that is mainly focused on the DevOps lifecycle and it is equally important as data and decisions should be made in such a way that the question should come to mind “Will this benefit the customer or their needs”?. Therefore gathering feedback from the customers on the already existing products will further guide the future optimization. Hence the DevOps team also makes use of live monitoring strategies to address the problems and uses other tools to allow them to measure how end users interact with the applications in real time. The speed of the DevOps lifecycle then further allows the teams to update which aims to remove these pain points.

3. Making Data Based Decision

Data-based decision-making is another DevOps principle which is used for informing decisions with data. Therefore whether selecting the right tech stack or selecting the various types of tools to streamline the pipeline, the developers should always collect the data all around each decision to make sure that the choices agree with the historical data and team metrics. Hence **having the knowledge of the average resolution time will help the developers to make informed decisions whenever introducing any new tools or procedures to the pipeline.** Developers can compare their results against the benchmarks and have an idea about the new addition which will help the team in the future.

4. Automation

The major benefit of the DevOps approach is the speed of the software delivery and the speed of the patches. This momentum is mainly achieved

with automation. **The DevOps team has an objective to automate every single phase of the process from the code reviews to handoff to provisioning and deployment.** This not only does allow the pipeline to move faster but it also leads to higher job satisfaction among the team members. Therefore they no longer need to perform manual and tedious tasks rather they can focus on higher order tasks like planning future improvements and researching new technologies.

5. Regular Improvement

The process of DevOps mainly focuses on a regular improvement and the development team should regularly focus on the new upgrades and features. The idea of Agile methodology mainly focuses on incremental releases. The past software development strategies aim to deliver the perfect product at once. Therefore in execution, it often means that the software deliveries would be delayed for long periods while the issues are been resolved. Incremental releases allow the teams to focus on achieving a minimum viable product to meet the customer's core use cases as soon as possible. Hence once the MVP is being delivered the team then shifts to producing features to add additional value to the products and works their way towards better software.

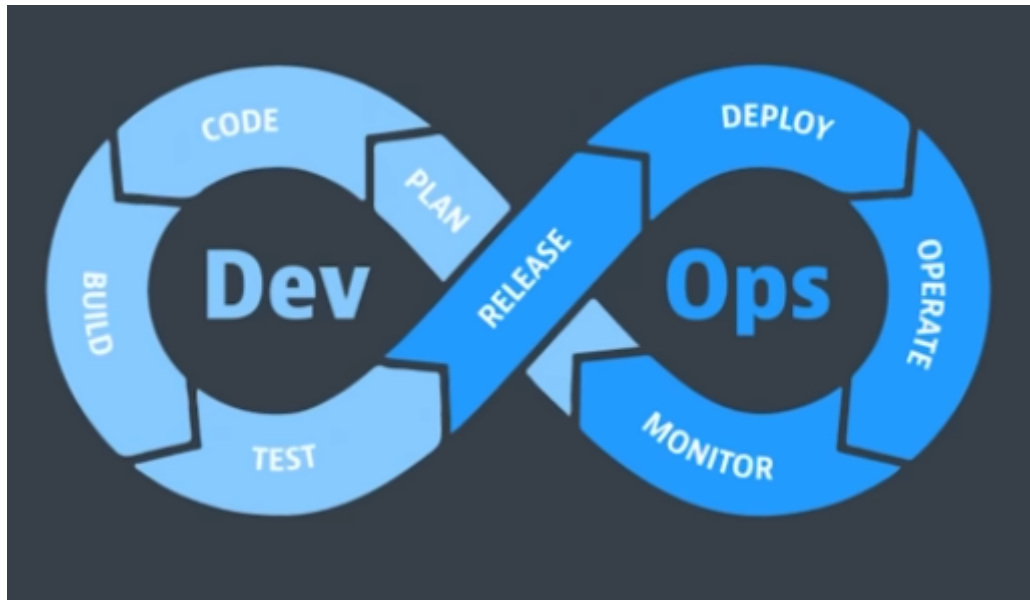
6. Failure as a Learning Opportunity

The process of development is getting better day by day as the software itself is continuously improving. **Thus the part of maintaining flexibility is to view the failure as an opportunity to learn and improve instead of trying to avoid the failure at any cost by encouraging risk-taking in the right context.** Therefore risks come with the possibility of failure but it can also lead to success, no matter how the experiment goes you can have the knowledge of what kind of work works. Hence this experience helps to plan future strategies and acts as another main point in decision making.

7. Responsibility Throughout the Lifecycle

Earlier in the past the software development models, the development team coded and developed the applications. Then they hand it to the operations teams to test, deploy, and deliver to the customer. Therefore if any bugs or errors are been discovered in the second phase are left to the operations teams rather than the software developers who wrote the code. Hence **DevOps shows a more logical approach and responsibility throughout the lifecycle.** The whole team is mainly responsible for the

product from the initial planning to its end. During the process, the operation and development teams are working hand in hand to update the software and address the issues.



Conclusion

DevOps embodies cultural philosophies and collaborative practices that aim to streamline the intricate software development and the deployment process. **DevOps principles are the essential practice to automate as much of the software development life cycle as possible** which provides the software developers more time to write code and create new features. Therefore, in this article, a comprehensive overview has been provided about the **DevOps principles** and the **top 7 DevOps principles** that are used for successful development teams.

What is CI/CD?

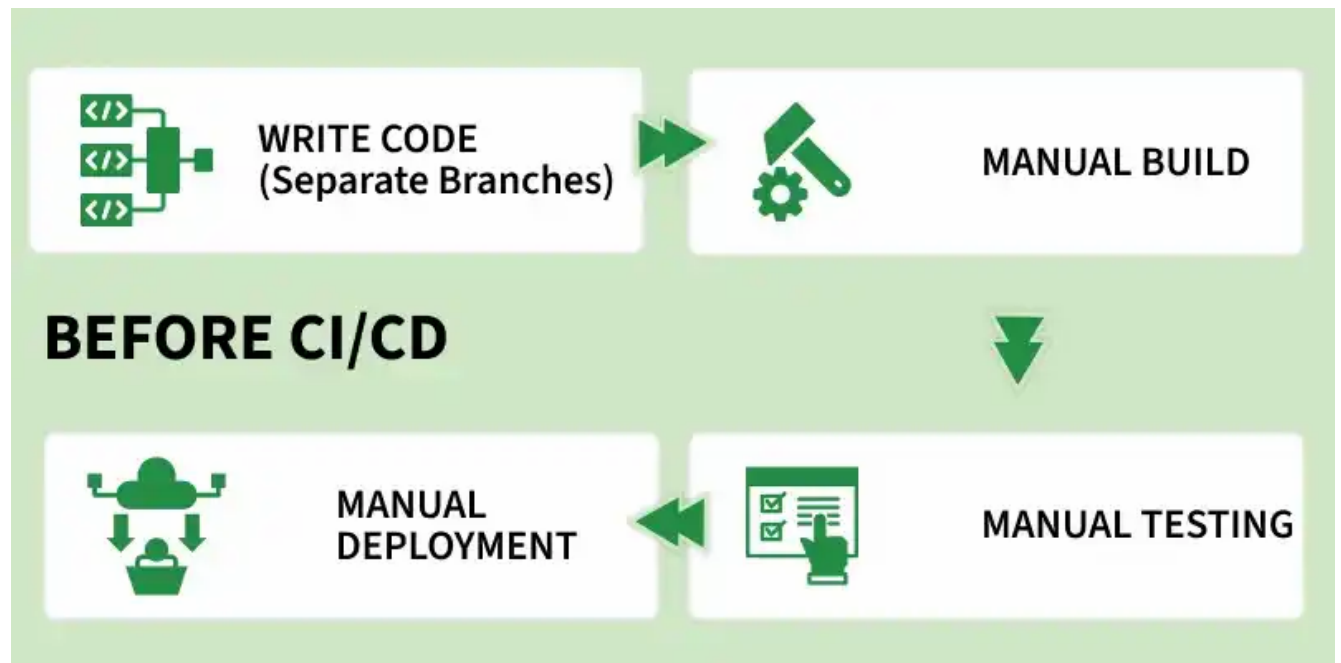
CI/CD stands for Continuous Integration and Continuous Delivery/Deployment, a practice that automates the process of building, testing, and releasing software.

It forms the backbone of modern DevOps by connecting development and operations to deliver applications faster and more reliably.

- It automates code integration, testing, and deployment workflows.
- Reduces manual effort while improving software quality.
- Enables faster, more consistent, and reliable releases.

Before CI/CD

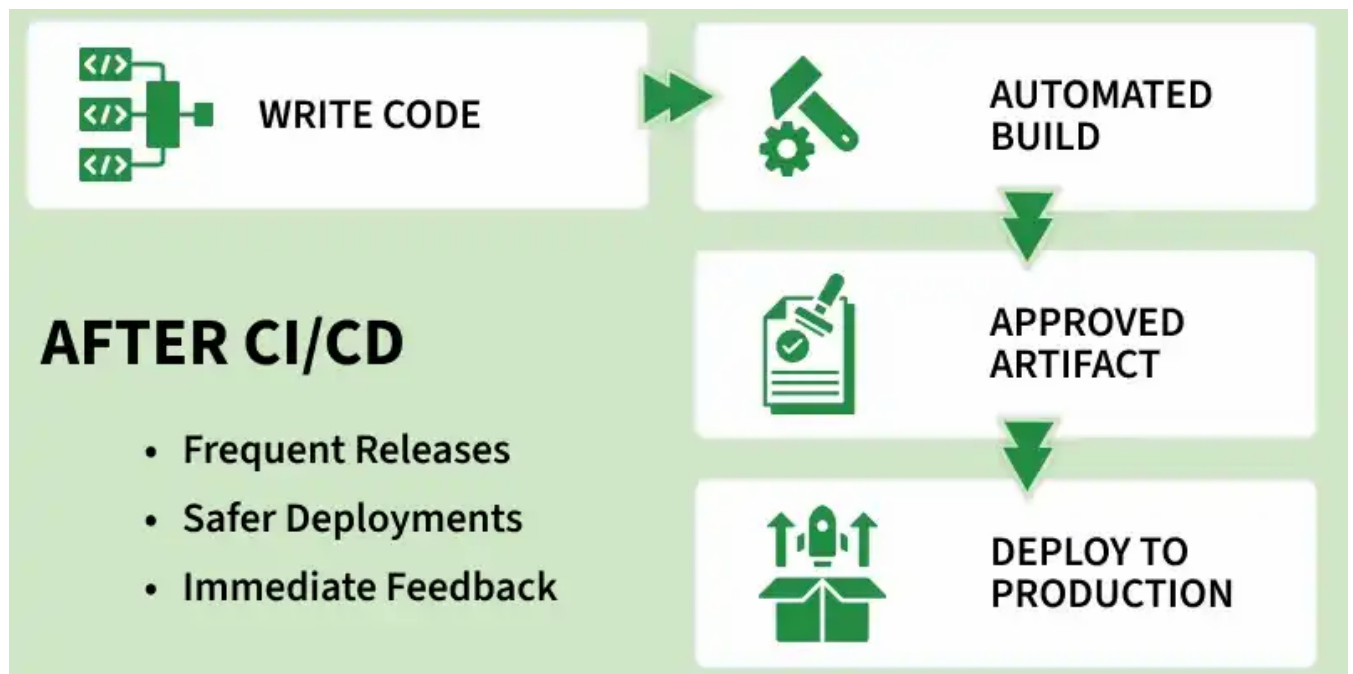
In the past, software delivery was slow and full of manual work. Developers used to create separate feature branches and worked on them for weeks or even months.



- All branches were merged at the end, which often caused big conflicts and broken builds. Testing and building the code were also manual, usually done at the final stage, so bugs were discovered very late and were costly to fix.
- Deployment was a long and risky process, taking days or sometimes weeks, because everything was released together in one big update.
- Teams were also divided: developers wrote code, testers tested it, and operations managed servers.

After CI/CD

With CI/CD, the whole process became faster, automated, and more reliable. Developers now commit their code frequently to a shared main branch, which prevents conflicts and ensures smooth integration.



- Automated pipelines run tests, builds, and checks as soon as code is pushed, so errors are caught early. Continuous Delivery makes sure that tested code is automatically packaged and ready for deployment.
- While Continuous Deployment takes it a step further by directly releasing to production without manual steps. This means smaller, frequent updates instead of one big risky release.
- Tools like Git, Jenkins, Docker, and Kubernetes automate everything from building to deployment, making the workflow transparent and collaborative. Bugs are fixed quickly, rollbacks are easier, and overall software delivery happens in hours instead of days.

The Three Pillars of CI/CD

To master CI/CD, you must understand the subtle but critical differences between the three "Continuous" concepts.

1. Continuous Integration (CI)

- **The Goal:** Prevent "Integration Hell."
- **The Process:** Developers merge their changes back to the main branch as often as possible (daily).
- **The Automation:** Upon every commit, an automated system builds the code and runs unit tests. If a test fails, the build is rejected, and the developer is notified immediately.

2. Continuous Delivery (CD)

- **The Goal:** Be ready to release at any moment.

- **The Process:** After CI passes, the code is automatically deployed to a staging/testing environment. Extensive integration and load tests are run.
- **The Release:** The codebase is always in a deployable state, but the **push to production is a manual decision** (e.g., a "Deploy" button click by a manager).

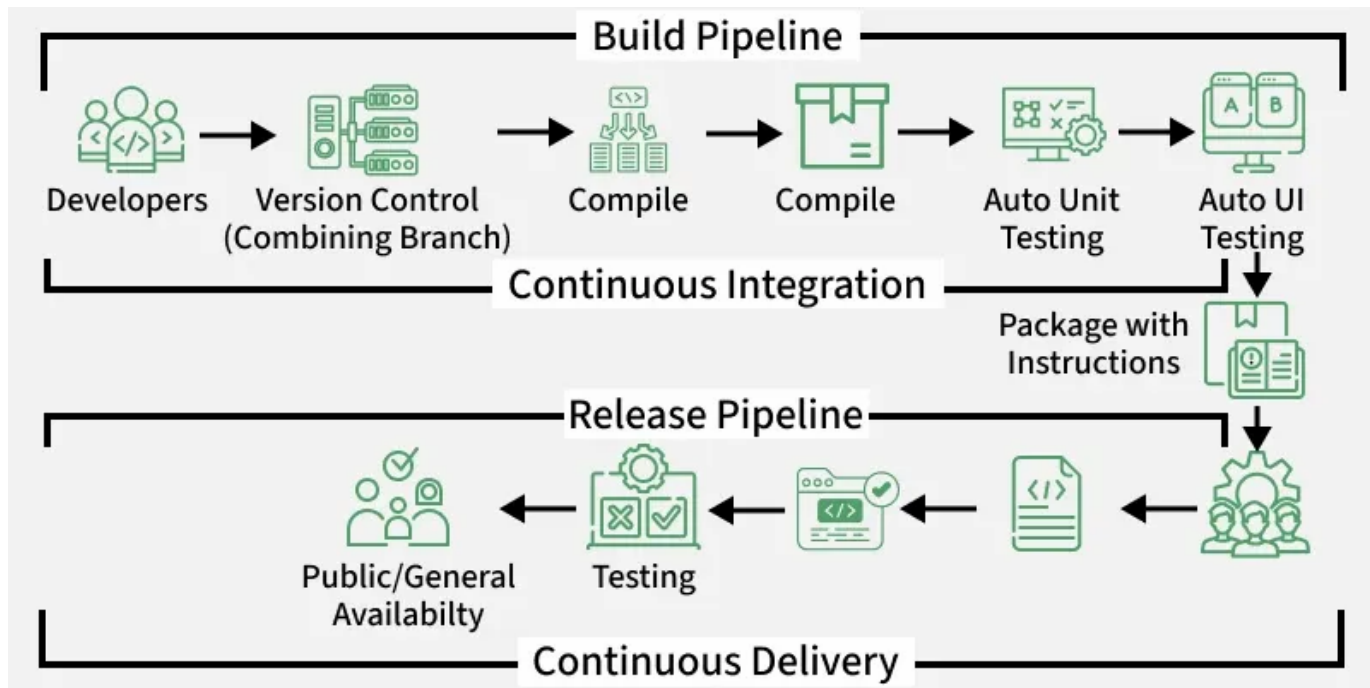
3. Continuous Deployment (CD)

- **The Goal:** Eliminate human intervention.
- **The Process:** Same as Continuous Delivery, but if all automated tests pass in staging, the code is **automatically deployed to production**.
- **The Requirement:** Requires extremely robust, trustworthy automated testing suites

What is CI/CD Pipeline?

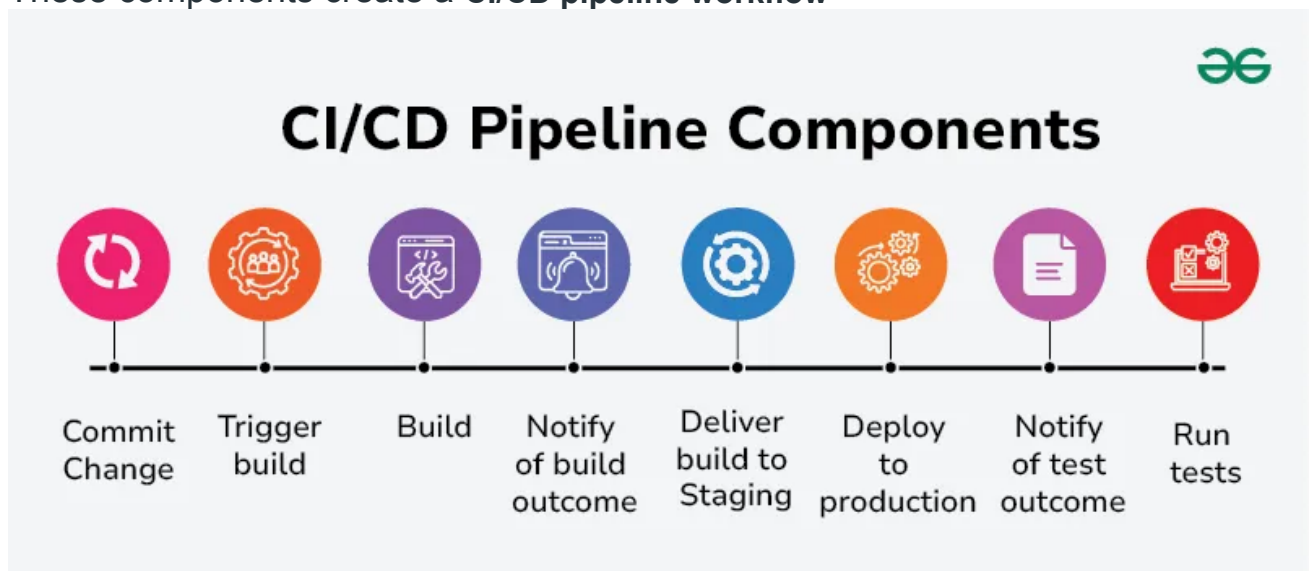
Many people think CI/CD and CI/CD Pipeline are the same, but they are a bit different.

- **CI/CD (Continuous Integration and Continuous Delivery):** This is a way of working where developers can make many code changes and deliver updates quickly.
 - **CI/CD Pipeline:** This is the actual step-by-step process that takes the code and automatically moves it from development to live production.
- In simple words, a CI/CD pipeline is a series of automatic steps that help developers build, test, and deliver code safely and quickly.



Components of a CI/CD Pipeline

These components create a CI/CD pipeline workflow



1. Commit Change

Developers make code changes and commit these changes to a version control system (e.g., Git). This step initiates the CI/CD pipeline. Committing code changes ensures that they are tracked and versioned properly.

2. Trigger Build

The version control system detects the new commit and triggers the build process automatically. Automated triggering of the build process ensures that new changes are continuously integrated and tested.

3. Build

The codebase is compiled and built into a deployable artifact, such as a binary executable or a Docker image. Common tools include Maven, Gradle, Ant for Java projects, and Docker for containerized applications. The build step verifies that the code compiles correctly and that all dependencies are resolved.

4. Notify of Build Outcome

The CI/CD system notifies the team of the build results, whether it passed or failed. Immediate feedback on the build status helps developers quickly identify and resolve any build issues. Notification systems can include emails, chat integrations (like Slack or Microsoft Teams), or dashboards in tools like Jenkins or GitLab CI.

5. Run Tests

Automated tests are executed on the build artifact. These can include unit tests, integration tests, end-to-end tests, and more. Testing frameworks like JUnit, Selenium, TestNG, pytest, etc. Running tests ensures that the new code does not introduce any bugs or regressions and that it meets the required quality standards.

6. Notify of Test Outcome

The results of the test suite are reported back to the development team. Quick feedback on test outcomes allows developers to address any failures promptly. Similar to build notifications, results can be sent via email, chat, or dashboards.

7. Deliver Build to Staging

If tests pass, the build artifact is deployed to a staging environment. The staging environment simulates the production environment, allowing for final validation before production deployment. Deployment tools like

Ansible, Chef, Puppet, Kubernetes, or cloud-specific services like AWS CodeDeploy.

8. Deploy to Production

After successful validation in staging, the build is automatically or manually promoted to the production environment. This step makes the new features and fixes available to end-users. Deployment strategies might include blue-green deployment, canary releases, or rolling updates to minimize downtime and risk.

Best Practices for a Healthy Pipeline

1. **Fast Feedback:** If a build fails, the developer should know within minutes, not hours.
1. **Commit Early and Often:** Large merge conflicts are the enemy of CI.
1. **Fix Broken Builds Immediately:** If the main branch is broken, no one else can merge. Fixing it is top priority.
1. **Environment Parity:** Staging should look exactly like Production. If they differ, your tests are unreliable.
1. **Infrastructure as Code (IaC):** Use tools like Terraform or CloudFormation to provision the environments your pipeline deploys to.

deployment strategies (blue-green, canary, rolling updates)

Microservices architecture has become a standard approach for building scalable and maintainable applications. However, deploying microservices presents unique challenges due to the distributed nature of these systems. This article explores three deployment strategies that address these challenges: Blue-Green, Canary, and Rolling Updates.

Blue-Green Deployment

Blue-Green deployment involves maintaining two identical production environments, dubbed “Blue” and “Green.” At any given time, only one environment serves production traffic.

The process typically follows these steps:

1. The current production environment (Blue) handles all user traffic.
2. A new version is deployed to the inactive environment (Green).
3. The Green environment undergoes testing and validation.
4. Once verified, traffic is switched from Blue to Green.
5. Green becomes the new production environment, and Blue becomes inactive.

Blue-Green deployment offers zero-downtime updates and quick rollback capabilities. However, it requires maintaining two full production environments, which can be resource-intensive.

Pros:

One of the benefits of the blue-green deployment is that it is simple, fast, well-understood, and easy to implement. Rollback is also straightforward, because you can simply flip traffic back to the old environment in case of any issues. Blue-green deployments are therefore not as risky compared to other deployment strategies.

Cons:

Cost is a drawback to blue-green deployments. Replicating a production environment can be complex and expensive, especially when working with microservices. Quality assurance and user acceptance testing may not identify all of the anomalies or regressions either, and so shifting all user traffic at once can present risks. An outage or issue could also have a wide-scale business impact before a rollback is triggered, and depending on the implementation, in-flight user transactions may be lost when the shift in traffic is made.

Canary Deployment

Canary deployment involves gradually rolling out changes to a small subset of users before a full deployment[2][5]. This approach allows teams to test new features in a production environment with minimal risk.

The process typically involves:

1. Deploying the new version alongside the current version.
2. Routing a small percentage of traffic to the new version.
3. Monitoring the new version's performance and user feedback.
4. Gradually increasing traffic to the new version if no issues are detected.

5. Fully transitioning to the new version once it's proven stable.

Canary deployment allows for early detection of issues and provides flexibility in rollout speed. However, it requires sophisticated traffic routing and monitoring capabilities.

Pros:

Canary deployments allow organizations to test in production with real users and use cases and compare different service versions side by side. It's cheaper than a blue-green deployment because it does not require two production environments. And finally, it is fast and safe to trigger a rollback to a previous version of an application.

Cons:

Drawbacks to canary deployments involve testing in production and the implementations needed. Scripting a canary release can be complex: manual verification or testing can take time, and the required monitoring and instrumentation for testing in production may involve additional research.

Rolling Updates

Rolling updates involve gradually replacing instances of the old version with the new version. This strategy is often used in container orchestration platforms like Kubernetes.

The process typically involves:

1. Taking down a subset of the old version instances.
2. Deploying new version instances in their place.
3. Waiting for the new instances to become ready.
4. Repeating the process until all instances are updated.

Rolling updates provide a balance between deployment speed and risk. They allow for gradual updates without requiring additional infrastructure. However, they can be slower than other strategies and may result in multiple versions running simultaneously.

Pros:

The benefits of a rolling deployment are that it is relatively simple to roll back, less risky than a basic deployment, and the implementation is simple.

Cons:

Since nodes are updated in batches, rolling deployments require services to support both new and old versions of an artifact. Verification of an application deployment at every incremental change also makes this deployment slow.

Choosing the Right Strategy

The choice of deployment strategy depends on various factors:

1. **Application Architecture:** Blue-Green and Canary deployments are well-suited for microservices architectures, while Rolling Updates can be used for both monolithic and microservices applications.
2. **Infrastructure:** Blue-Green requires additional resources to maintain two environments. Canary and Rolling Updates can be implemented with existing infrastructure.
3. **Risk Tolerance:** Canary deployments offer the lowest risk by limiting exposure to a small user base. Blue-Green provides quick rollback capabilities, while Rolling Updates balance risk and speed.
4. **Deployment Frequency:** For frequent updates, Rolling Updates or Canary deployments may be more efficient. Blue-

Green can be resource-intensive for high-frequency deployments.

5. **Monitoring Capabilities:** Canary deployments require sophisticated monitoring to detect issues in the new version quickly.

Implementation Considerations

Regardless of the chosen strategy, consider these implementation aspects:

1. **Automation:** Use CI/CD pipelines to automate the deployment process. Tools like Jenkins, GitLab CI, or CircleCI can help orchestrate deployments.
2. **Configuration Management:** Use tools like Ansible, Puppet, or Chef to manage configuration across environments.
3. **Service Discovery:** Implement service discovery mechanisms to handle dynamic IP addresses and ports in microservices environments.
4. **Database Schema Changes:** Coordinate database updates with application deployments to ensure compatibility.
5. **Monitoring and Logging:** Implement comprehensive monitoring and logging to quickly identify and troubleshoot issues during and after deployment.
6. **Traffic Management:** Use tools like Istio or NGINX for fine-grained traffic control, especially for Canary deployments.

7. **Rollback Plan:** Always have a rollback strategy in place, regardless of the chosen deployment method.

In conclusion, Blue-Green, Canary, and Rolling Updates each offer unique advantages for microservices deployment. By understanding these strategies and considering the specific needs of your application and organization, you can implement a deployment approach that balances risk, speed, and resource utilization effectively.

